

BIG DATA Y MACHINE LEARNING



Introducción al web scraping

Big Data and Machine Learning para Economía Aplicada

AUTORES/AS

Julián David Rojas Aguilar
Ignacio Sarmiento-Barbieri

AFILIACIÓN

Universidad de los Andes
Universidad de los Andes

FECHA DE PUBLICACIÓN

2026-08-21

El *web scraping*¹ consiste en descargar un documento HTML, interpretar su estructura mediante un *parser*², y extraer los elementos que contienen los datos de interés. Esto permite automatizar la extracción de datos y generar resultados reproducibles, pero debe reportarse el momento en que se descargaron los datos mediante este procedimiento, pues la página puede cambiar con el paso del tiempo.

Al finalizar esta sesión, deberás:

- reconocer los elementos y atributos básicos de un documento HTML.
- seleccionar elementos mediante el selector de CSS y XPath.
- extraer texto, atributos y tablas con `rvest`.
- recorrer las páginas de un sitio con múltiples páginas.
- reconocer cuándo una página carga sus datos mediante una solicitud adicional.
- extraer elementos dinámicos dentro de una página.

Reconocimiento: La mayoría de los materiales presentados aquí fueron aportados por profesores complementarios anteriores, especialmente [Eduard-Martínez](#), [Lucas Gómez](#), Andrés Rengifo, Valentina Laverde, Juan Esteban Segura y Gustavo Castillo. Estamos verdaderamente agradecidos por su ayuda.

Configuración inicial

▼ Mostrar/ocultar el código

```
1 library(pacman)
2
```

```
3 p_load(  
4   tidyverse, # Manipulación de datos.  
5   rvest,     # Web scraping.  
6   httr,      # Solicitud de datos para páginas dinámicas.  
7   jsonlite  # Lectura de datos en formato .json.  
8 )
```

¿En qué fijarnos al hacer *web scraping*?

- **¿Existe una opción de descarga directa o una API?** Distintas entidades exhiben datos en sus páginas web. En algunos casos ofrecen una descarga directa, por lo que somos capaces de obtener, fácilmente, archivos en formato `.csv`, `.dta`, `.json`, etc. Si no, es posible que hayan diseñado una *Application Programming Interface* (API) para acceder a los datos mediante algún lenguaje de programación –como, por ejemplo, `R`–. Esta API entrega la información de forma estructurada a nuestro entorno y suele ser una alternativa estable. Sin embargo, este no siempre es el caso. Cuando no es posible, el *web scraping* permite automatizar la extracción de información publicada en sitios web³.
- **¿Es confiable la información que estamos descargando?** Como cualquier persona puede publicar datos en Internet, debemos tener cuidado cuando hacemos *web scraping*. Por ejemplo, verificando que sea una fuente oficial.
- **¿Qué tenemos permitido *scrapear*? ¿Bajo qué términos?** Debemos consultar las reglas del sitio y evitar solicitudes innecesariamente frecuentes.

Robots.txt

El *Robots Exclusion Protocol*, conocido por el archivo `robots.txt`, permite que un sitio **comunique** a los *scrapers* qué rutas pueden o no pueden consultar. Estas reglas orientan el comportamiento de los robots, pero no constituyen una restricción en términos legales estrictamente. Su cumplimiento es, por tanto, de carácter ético. Lo correcto sería revisar los términos de uso del sitio y respetar las restricciones indicadas.

Veamos algunos ejemplos de `robots.txt`:

Ejemplo 1. Permite a cualquier robots procesar/escanear todos los elementos de la página.

```
# robots.txt for https://example.com/  
User-agent: *  
Disallow:
```

Ejemplo 2. No permite procesar/escanear ningún elemento de la página. Note que cualquier elemento desde la raíz del árbol está restringida.

```
# robots.txt for https://example.com/
```

```
User-agent: *
```

```
Disallow: /
```

Ejemplo 3. No permite procesar/escanear ningún elemento del archivo /public/index.html.

```
# robots.txt for https://example.com/
```

```
User-agent: *
```

```
Disallow: /public/index.html
```

Ejemplo 4. No le permite al usuario/cliente **BadBot** procesar/escanear ningún elemento de la página.

```
# robots.txt for https://example.com/
```

```
User-agent: BadBot
```

```
Disallow: /
```

Puede acceder al archivo agregando `/robots.txt` al dominio principal. Por ejemplo:

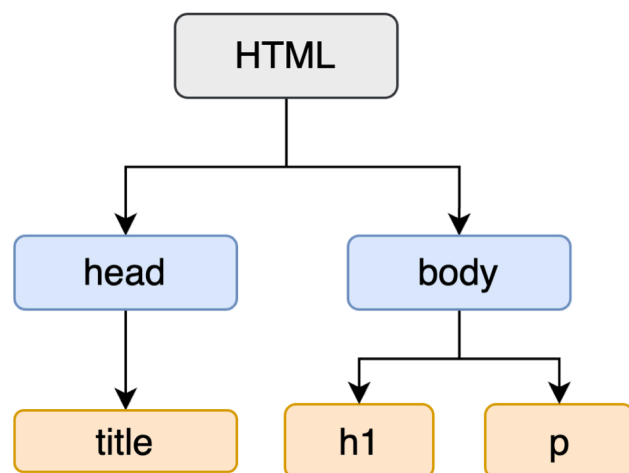
<https://example.com/robots.txt>. Consulte los archivos de [Wikipedia](#), [Win Sports](#), [Banco de la República](#) y [Facebook](#).

HyperText Markup Language (HTML)

HTML (*HyperText Markup Language*) no es un lenguaje de programación, sino un lenguaje de marcado —como LaTeX o Markdown—. Un documento HTML está compuesto por **elementos**. Cada elemento puede contener una etiqueta de apertura, una etiqueta de cierre, atributos y contenido. El navegador interpreta estos elementos como un árbol: algunos nodos contienen otros nodos y establecen una estructura jerárquica. Para extraer información debemos identificar el nodo que la contiene.

Representación de un documento HTML como un árbol de nodos.

```
<!DOCTYPE html>
<html>
  <head>
    <title></title>
  </head>
  <body>
    <h1></h1>
    <p></p>
  </body>
</html>
```



Los navegadores interpretan el HTML y muestran su contenido como una página web. Puede inspeccionar el HTML de una página con las herramientas de desarrollo del navegador:

Cómo abrir el inspector de elementos del navegador.



Elementos en un HTML

Un elemento está compuesto por una etiqueta (`<p>`), atributos opcionales (`id="saludo"`) y contenido (`Hola, mundo`). Por ejemplo:

```
<p id="saludo">Hola, mundo</p>
```

Las etiquetas delimitan el inicio (`<p>`) y el final (`</p>`) de un elemento, aunque algunos elementos no requieren una etiqueta de cierre. Estas son algunas etiquetas comunes:

- `<body>`: Cuerpo de la página.
- `<head>`: Encabezado de la página.
- `<h1>`, `<h2>`, ..., `<h6>`: Encabezados.
- `<div>`: División. Sirve para crear secciones o agrupar contenidos.
- `<p>`: Párrafos.
- `<a>`: Enlaces.
- `<li>`: Ítem en una lista. La lista puede ser ordenada, `<ol>` (de *ordered list*), o desordenadas, `<ul>` (de *unordered list*).
- `<table>`: Tablas.
- `<td>`: Una celda de datos en una tabla.

Los atributos agregan información a un elemento. Se escriben dentro de la etiqueta de apertura como pares de nombre y valor. Por ejemplo:

```
<a class="document-toc-link" href="atributos.html">Lista de atributos</a>
```

La etiqueta `<a>` crea un enlace. El atributo `class` permite agrupar o identificar elementos con características comunes, mientras que `href` contiene la dirección de destino del enlace.

Mi primer HTML

Con lo expuesto hasta el momento, podemos escribir nuestro primer **HTML**. Vamos a escribirlo a través de `R`.

▼ Mostrar/ocultar el código

```
1 my_html <- '<!DOCTYPE html>
2 <html>
3 <head>
4   <meta charset="utf-8">
5   <title>Ejemplo de clase</title>
6 </head>
7 <body>
8   <h1>Título principal</h1>
9   <h2>Subtítulo <u>subrayado</u></h2>
10  <p>Este párrafo está dentro de una etiqueta <b>p</b> de <i>HTML</i>.</p>
11 </body>
12 </html>'
```

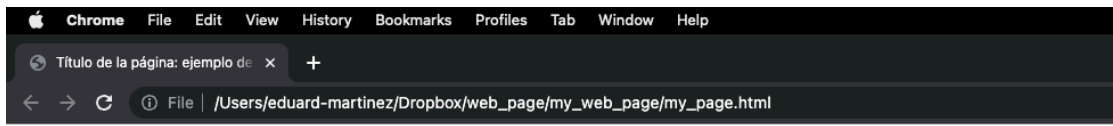
Ahora guardará el objeto `my_html` como un archivo `.html`:

▼ Mostrar/ocultar el código

```
1 writeLines(
2   text = my_html,
3   con = "my_page.html"
4 )
```

Después de ejecutar el código, abra [el archivo HTML generado](#) en su navegador. Su navegador interpretará el archivo `my_page.html` y debe visualizar lo siguiente:

Resultado esperado al abrir el primer documento HTML.



Title 1.

Subtitle subrayado-1.

Este es un párrafo muy pequeño que se encuentra dentro de la etiqueta **p** de *html*

Nota. Modifique el objeto `my_html`: agregue un elemento `<h1>` con su nombre después del párrafo. Guarde nuevamente el archivo y observe cómo cambia la página.

Librería `rvest`

`rvest` contiene funciones para recolectar datos desde páginas web **estáticas**: páginas cuyo contenido de interés está presente en el HTML que descargamos. Puede consultar la [introducción oficial a `rvest`](#) como material adicional.

El procedimiento básico tiene tres pasos:

1. descargar e interpretar el HTML con `read_html()`.
2. seleccionar los elementos que contienen la información mediante `html_elements()`, y
3. extraer su texto, `html_text2()`, sus atributos, `html_attrs()`, o sus tablas, `html_table()` –dependiendo de qué necesites–.

Aplicaremos este procedimiento en dos ejemplos: Wikipedia y *Quotes to Scrape*, una página empleada para el aprendizaje de *web scraping*. Otra página interesante que podrías explorar es [Books to Scrape](#).

Ejemplo 1: Wikipedia

Comenzaremos con la página de Wikipedia sobre la [Copa Mundial de Fútbol](#). La función `read_html()` descarga el documento (es decir, una cadena de texto) y lo interpreta/representa como un árbol.

▼ Mostrar/ocultar el código

```

1 world_cup_url <- "https://es.wikipedia.org/wiki/Copa_Mundial_de_F%C3%BAtbo1"
2
3 world_cup_html <- read_html(world_cup_url)
4 class(world_cup_html)

```

```
[1] "xml_document" "xml_node"
```

Representación de un documento HTML importado en R.

Name	Type	Value
<html>	list [2] (S3: xml_document, xr	List of length 2
<head>	list [2] (S3: xml_node)	List of length 2
<body>	list [2] (S3: xml_node)	List of length 2
(xml_attributes)	character [3]	'client-nojs' 'es' 'ltr'

Seleccionar elementos con CSS

Los **selectores CSS** describen qué elementos queremos recuperar mediante una sintaxis en particular. Los más útiles para comenzar son:

Selector	¿Qué elige?
<code>p</code>	todos los elementos <code><p></code>
<code>.quote</code>	todos los elementos con <code>class="quote"</code>
<code>#content</code>	el elemento con <code>id="content"</code>
<code>.quote .author</code>	los elementos de clase <code>author</code> dentro los elementos de clase <code>quote</code>
<code>a[href]</code>	los enlaces que tienen el atributo <code>href</code>

`html_elements()` devuelve todos los elementos que coinciden con un selector. Por ejemplo, los elementos `<h2>` contienen los encabezados de segundo nivel:

▼ Mostrar/ocultar el código

```

1 world_cup_html |>
2   html_elements("h2") |>
3   html_text2()

```

```

[1] "Contenidos"      "Historia"
[3] "Desarrollo"      "Resultados y estadísticas"
[5] "Premios"         "Impacto cultural"
[7] "Tecnología"      "Véase también"

```

[9] "Notas"	"Referencias"
[11] "Bibliografía"	"Enlaces externos"

Cuando solo necesitamos la primera coincidencia, usamos `html_element()`. El siguiente selector busca el primer párrafo dentro del contenido principal:

▼ Mostrar/ocultar el código

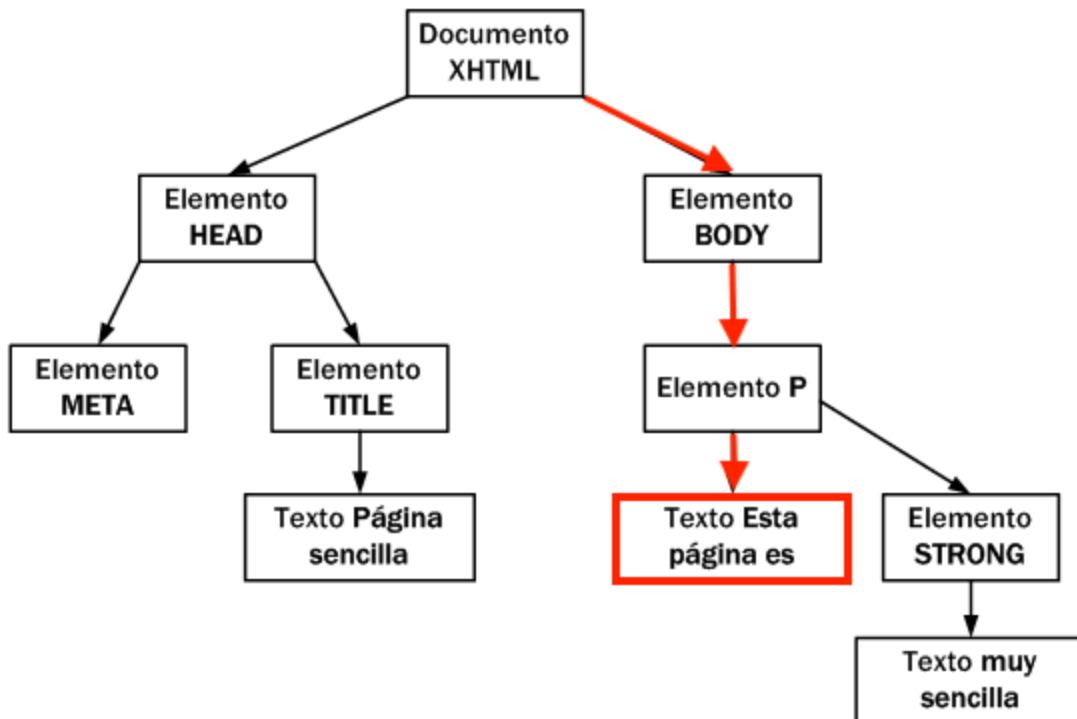
```
1 world_cup_html |>  
2   html_element("p") |>  
3   html_text2()
```

```
[1] "La Copa Mundial de la FIFA, también conocida como Copa Mundial de Fútbol, Copa del Mundo o simplemente Mundial, es el principal torneo internacional oficial de fútbol masculino a nivel de selecciones nacionales en el mundo.[n 1]"
```

Seleccionar elementos con XPath

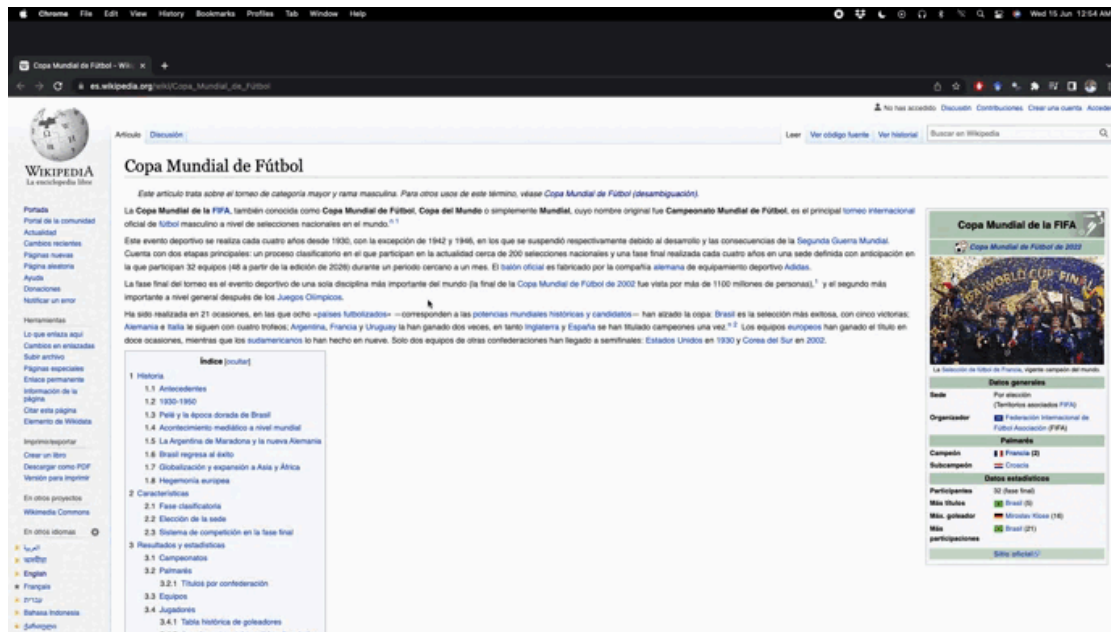
XPath (*XML Path Language*) es otra forma de recorrer el árbol de un documento. Suele ser útil cuando necesitamos expresar relaciones jerárquicas o condiciones más específicas. En general, conviene comenzar con selectores CSS porque son más fáciles de entender y suficientes para muchas tareas, pero el XPath permite alcanzar nodos que, con CSS, no es posible.

Ubicación de un elemento dentro del árbol HTML.



El navegador permite copiar el XPath de un elemento desde el inspector:

Cómo copiar el XPath desde el inspector del navegador.



El siguiente XPath selecciona el primer párrafo que aparece dentro del elemento con `id="mw-content-text"`. Note que, cuando utilizamos XPath, es necesario especificar el argumento `xpath -o`, de lo contrario, se generará un error-.

▼ Mostrar/ocultar el código

```
1 world_cup_html |>
2   html_element(xpath = "//*[@id='mw-content-text']//p") |>
3   html_text2()
```

```
[1] "La Copa Mundial de la FIFA, también conocida como Copa Mundial de Fútbol, Copa del Mundo o simplemente Mundial, es el principal torneo internacional oficial de fútbol masculino a nivel de selecciones nacionales en el mundo.[n 1]"
```

En esta expresión, `//*` permite buscar cualquier elemento. `[@id='...']` es un predicado que restringe la búsqueda del elemento `*` a aquel con un atributo particular: su `id` debe ser `mw-content-text`. Finalmente, `//p` busca todos los párrafos descendientes. Otros patrones frecuentes son:

- `//*[name()='abc']`: selecciona elementos cuya etiqueta es `<abc>`.
- `//*[text()='abc']`: selecciona elementos cuyo texto es `abc`.
- `//*[@id='abc']`: selecciona el elemento con `id="abc"`.

Para más información, véase el capítulo 4 de Munzert et al. (2015).

Extraer tablas

`html_table()` convierte una tabla HTML en un *tibble*. En lugar de escoger una tabla por su posición — que puede cambiar—, primero seleccionamos la clase `wikitable` utilizada por Wikipedia:

▼ Mostrar/ocultar el código

```
1 world_cup_table <- world_cup_html |>
2   html_element("table.wikitable") |>
3   html_table()
4
5 world_cup_table
```

Año	Sede	Costo (en miles de millones USD)	Fuente
<int>	<chr>	<dbl>	<chr>
1994	Estados Unidos	0.5	[124]
1998	Francia	2.3	[124]
2002	Corea Japón	7.0	[124]
2006	Alemania	4.3	[124]
2010	Sudáfrica	3.6	[124]
2014	Brasil	15.0	[124]
2018	Rusia	11.6	[124]
2022	Catar	220.0	[124]

8 rows

Por temas pedagógicos, conviene explorar cuál sería la redacción en XPath.

▼ Mostrar/ocultar el código

```
1 world_cup_table <- world_cup_html |>
2   html_element(xpath = "//table[@class = 'wikitable']") |>
3   html_table()
4
5 world_cup_table
```

Año	Sede	Costo (en miles de millones USD)	Fuente
<int>	<chr>	<dbl>	<chr>
1994	Estados Unidos	0.5	[124]
1998	Francia	2.3	[124]
2002	Corea Japón	7.0	[124]
2006	Alemania	4.3	[124]
2010	Sudáfrica	3.6	[124]
2014	Brasil	15.0	[124]
2018	Rusia	11.6	[124]
2022	Catar	220.0	[124]

8 rows

Extraer atributos

El texto visible de un enlace y su dirección son objetos distintos. El texto se extrae con `html_text2()`, mientras que `html_attr("href")` recupera el atributo que contiene la dirección⁴.

▼ Mostrar/ocultar el código

```
1 link_nodes <- world_cup_html |>
2   html_elements("#mw-content-text a[href]")
3
4 world_cup_links <- tibble(
5   text = link_nodes |> html_text2(),
6   url = link_nodes |> html_attr("href") |> url_absolute(world_cup_url)
7 )
8
9 world_cup_links |> head()
```

text

<chr>

Copa Mundial de Fútbol (desambiguación)

Territorios asociados a la FIFA

Federación Internacional de Fútbol Asociación

TV Pública

6 rows | 1-1 of 2 columns

Aplicación: partidos de fútbol

[Esta página de Wikipedia](#) publica los resultados de los 200 partidos de la fase de todos contra todos del Torneo Apertura de 2017. En el HTML, cada partido está almacenado como una tabla con la clase `vevent`.

▼ Mostrar/ocultar el código

```
1 football_url <- "https://es.wikipedia.org/wiki/Anexo:Torneo_Apertura_2017_(Colombia)_-_Fase_
2
3 football_html <- read_html(football_url)
4
5 match_tables <- football_html |>
6   html_elements("table.vevent") |>
7   html_table()
8
9 length(match_tables)
```

[1] 200

La primera fila de cada tabla contiene la fecha, los equipos, el marcador y el estadio. La segunda contiene información complementaria, como la asistencia y el árbitro:

▼ Mostrar/ocultar el código

```
1 match_tables[[1]]
```

X1	X2	X3
<chr>	<chr>	<chr>
3 de febrero de 2017, 19:45	Cortuluá	0:4 (0:1)
		Reporte

2 rows | 1-3 of 6 columns

Como todas las tablas comparten esa organización, podemos escribir una función que transforme cada una en una fila.

▼ Mostrar/ocultar el código

```
1 extract_match <- function(match_table) {
2
3   tibble(
4     date = match_table[["X1"]][1],
5     home_team = match_table[["X2"]][1],
6     score = match_table[["X3"]][1],
7     away_team = match_table[["X4"]][1],
8     stadium = match_table[["X5"]][1]
9   ) |>
10   mutate(across(everything(), str_squish))
11 }
```

Finalmente, `map_dfr()` aplica la función a cada elemento de la lista `match_tables` y combina los resultados en una sola base de datos:

▼ Mostrar/ocultar el código

```
1 football_data <- match_tables |>
2   map_dfr(extract_match)
3
4 football_data
```

date	home_team	score
<chr>	<chr>	<chr>
3 de febrero de 2017, 19:45	Cortuluá	0:4 (0:1)
4 de febrero de 2017, 15:15	Jaguars	1:0 (0:0)
4 de febrero de 2017, 17:00	América de Cali	0:0
4 de febrero de 2017, 17:00	Alianza Petrolera	1:2 (0:1)

date <chr>	home_team <chr>	score <chr>
4 de febrero de 2017, 19:45	La Equidad	1:0 (1:0)
5 de febrero de 2017, 15:00	Envigado F. C.	2:0 (1:0)
5 de febrero de 2017, 15:15	Atlético Huila	1:1 (1:0)
5 de febrero de 2017, 17:30	Once Caldas	0:0
5 de febrero de 2017, 19:30	Millonarios	1:2 (1:0)
29 de marzo de 2017, 18:00	Atlético Nacional	3:1 (2:0)

1-10 of 200 rows | 1-3 of 5 columns

Previous **1** 2 3 4 5 6 ... 20 Next

Ejemplo 2: Quotes to Scrape

[Quotes to Scrape](#) fue diseñada para practicar técnicas de *web scraping*. Cada recuadro contiene una cita, su autor, varias etiquetas temáticas y un enlace hacia la página del autor.

▼ Mostrar/ocultar el código

```
1 quotes_url <- "https://quotes.toscrape.com/"
2 quotes_html <- read_html(quotes_url)
```

Citas y autores publicados en Quotes to Scrape.

Quotes to Scrape

[Login](#)

"The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking."

by [Albert Einstein](#) (about)

Tags: [change](#) [deep-thoughts](#) [thinking](#) [world](#)

"It is our choices, Harry, that show what we truly are, far more than our abilities."

by [J.K. Rowling](#) (about)

Tags: [abilities](#) [choices](#)

"There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle."

by [Albert Einstein](#) (about)

Tags: [inspirational](#) [life](#) [live](#) [miracle](#) [miracles](#)

"The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid."

by [Jane Austen](#) (about)

Tags: [aliteracy](#) [books](#) [classic](#) [humor](#)

Top Ten tags

[love](#)
[inspirational](#)
[life](#)
[humor](#)
[books](#)
[reading](#)
[friendship](#)
[travel](#)
[truth](#)
[love](#)

Extraer observaciones repetidas

Cada cita está contenida en un elemento `<div class="quote">`. Por eso, la clase `.quote` identifica la **unidad de observación**: cada nodo seleccionado se convertirá en una fila de la base de datos.

▼ Mostrar/ocultar el código

```
1 quote_nodes <- quotes_html |>
2   html_elements(".quote")
3
4 quote_nodes
```

```
{xml_nodeset (10)}
```

```
[1] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
[2] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
[3] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
[4] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
[5] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
[6] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
[7] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
[8] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
[9] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
[10] <div class="quote" itemscope itemtype="http://schema.org/CreativeWork">\ ...
```

A partir de cada nodo, `html_element()` extrae una variable. Este procedimiento mantiene juntas la cita, la autora o el autor, las etiquetas y la dirección que pertenecen a una misma observación:

▼ Mostrar/ocultar el código

```
1 quotes_page_1 <- tibble(
2   quote = quote_nodes |>
3     html_element(".text") |>
4     html_text2(),
5   author = quote_nodes |>
6     html_element(".author") |>
7     html_text2(),
8   author_url = quote_nodes |>
9     html_element("a[href]") |>
10    html_attr("href") |>
11    url_absolute(quotes_url)
12 )
13
14 quotes_page_1
```

quote

<chr>

"The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking."

"It is our choices, Harry, that show what we truly are, far more than our abilities."

"There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle."

"The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid."

quote

<chr>

"Imperfection is beauty, madness is genius and it's better to be absolutely ridiculous than absolutely boring."

"Try not to become a man of success. Rather become a man of value."

"It is better to be hated for what you are than to be loved for what you are not."

"I have not failed. I've just found 10,000 ways that won't work."

"A woman is like a tea bag; you never know how strong it is until it's in hot water."

"A day without sunshine is like, you know, night."

1-10 of 10 rows | 1-1 of 3 columns

Recorrer varias páginas

Una página web con múltiples páginas distribuye sus observaciones entre varias direcciones. Quotes to Scrape muestra diez citas por página y utiliza el enlace contenido en [li.next a](#) para avanzar. La última página no contiene ese enlace, y esa ausencia nos indica cuándo detener el recorrido.

Primero creamos una función que extrae las citas de una página y devuelve la dirección de la siguiente:

▼ Mostrar/ocultar el código

```
1  scrape_quotes_page <- function(page_url) {
2    page_html <- read_html(page_url)
3
4    quote_nodes <- page_html |>
5      html_elements(".quote")
6
7    page_data <- tibble(
8      quote = quote_nodes |>
9        html_element(".text") |>
10       html_text2(),
11      author = quote_nodes |>
12        html_element(".author") |>
13        html_text2(),
14      author_url = quote_nodes |>
15        html_element("a[href]") |>
16        html_attr("href") |>
17        url_absolute(page_url)
18    )
19
20    next_path <- page_html |>
21      html_element("li.next a") |>
22      html_attr("href")
23  }
```

```

24   next_url <- if (is.na(next_path)) {
25     NA_character_
26   } else {
27     url_absolute(next_path, page_url)
28   }
29
30   list(
31     data = page_data,
32     next_url = next_url
33   )
34 }

```

Luego usamos un ciclo `while`. Mientras exista una dirección siguiente, el ciclo extrae una página, guarda sus observaciones y avanza. La pausa breve entre solicitudes evita enviar todas las peticiones de manera inmediata:

▼ Mostrar/ocultar el código

```

1  current_url <- quotes_url
2  quotes_by_page <- list()
3
4  while (!is.na(current_url)) {
5    page_result <- scrape_quotes_page(current_url)
6
7    quotes_by_page[[length(quotes_by_page) + 1]] <- page_result$data
8    current_url <- page_result$next_url
9
10   if (!is.na(current_url)) {
11     Sys.sleep(0.5)
12   }
13 }
14
15 quotes_data <- bind_rows(quotes_by_page)
16 quotes_data

```

quote

<chr>

"The world as we have created it is a process of our thinking. It cannot be changed without changing our thinking."

"It is our choices, Harry, that show what we truly are, far more than our abilities."

"There are only two ways to live your life. One is as though nothing is a miracle. The other is as though everything is a miracle."

"The person, be it gentleman or lady, who has not pleasure in a good novel, must be intolerably stupid."

"Imperfection is beauty, madness is genius and it's better to be absolutely ridiculous than absolutely boring."

"Try not to become a man of success. Rather become a man of value."

quote

<chr>

"It is better to be hated for what you are than to be loved for what you are not."

"I have not failed. I've just found 10,000 ways that won't work."

"A woman is like a tea bag; you never know how strong it is until it's in hot water."

"A day without sunshine is like, you know, night."

1-10 of 100 rows | 1-1 of 3 columns

Previous **1** 2 3 4 5 6 ... 10 Next

Páginas dinámicas

Una página dinámica no incluye, necesariamente, todos sus datos en el HTML inicial. La página misma, mediante JavaScript, puede realizar solicitudes adicionales e insertar sus resultados en la página. `read_html()` solo descarga el HTML: no ejecuta ese JavaScript. Por tanto, podemos observar datos en la página que no aparecen en el HTML.

Para ejemplificar esto, vamos a recuperar la tabla de posiciones de la Liga Dimayor que muestra [Win Sports](#) .

▼ Mostrar/ocultar el código

```
1 winsports_html <- read_html("https://www.winsports.co/")
```

Tabla de posiciones mostrada en la página de Win Sports.

The screenshot shows the website winsports.co. At the top, there are navigation links for various teams: JUNIOR, LIGA BETPLAY, ATLÉTICO NACIONAL, MILLONARIOS, SANTA FE, and AMÉRICA DE CALI. On the left, a sidebar contains links for FÚTBOL COLOMBIANO, TABLA DE POSICIONES, FÚTBOL INTERNACIONAL, CURIOSIDADES DEL DEPORTE, SELECCIÓN COLOMBIA, MÁS CONTENIDO, PROGRAMACIÓN, CAV-SULAS, MÁS DEPORTES, and PROGRAMAS. Below these is a search bar and a 'SUSCRÍBASE' button.

The main content area features a large video player titled 'VIDEOS' with a play button icon. Below the video, the text reads: 'FÚTBOL COLOMBIANO', 'Nacional venció a América en otra gran noche de Juan Rengifo', and 'Atlético Nacional se impuso 2-1 ante América de Cali y logró escalar posiciones en la tabla.' Below the video are three smaller video thumbnails.

On the right, there is a section titled 'POSICIONES' for 'LIGA DIMAYOR'. It displays a table with the following data:

EQUIPOS	PTS	PJ	DI
1 Pasto	9	4	
2 Bucaramanga	8	4	
3 Tolima	8	4	
4 América	7	4	
5 Inter Bogotá	7	4	
6 Atl. Nacional	6	2	
7 Cali	6	4	
8 Llaneros	6	4	
9 Once Caldas	6	4	
10 Santa Fe	6	4	
11 Fortaleza	6	4	
12 Junior	6	3	

Below the table, it indicates 'Cuartos de Final'.

Utilicemos lo aprendido hasta ahora. Vamos a extraer las tablas presentes en el HTML inicial:

▼ Mostrar/ocultar el código

```
1 winsports_html |>
2   html_table()
```

```
list()
```

Obtenemos una lista vacía porque la tabla no está en el documento descargado. El navegador la incorpora después de ejecutar JavaScript y recibir una respuesta a su solicitud. Estas solicitudes suelen aparecer en las herramientas de desarrollo bajo las categorías **Fetch/XHR**.

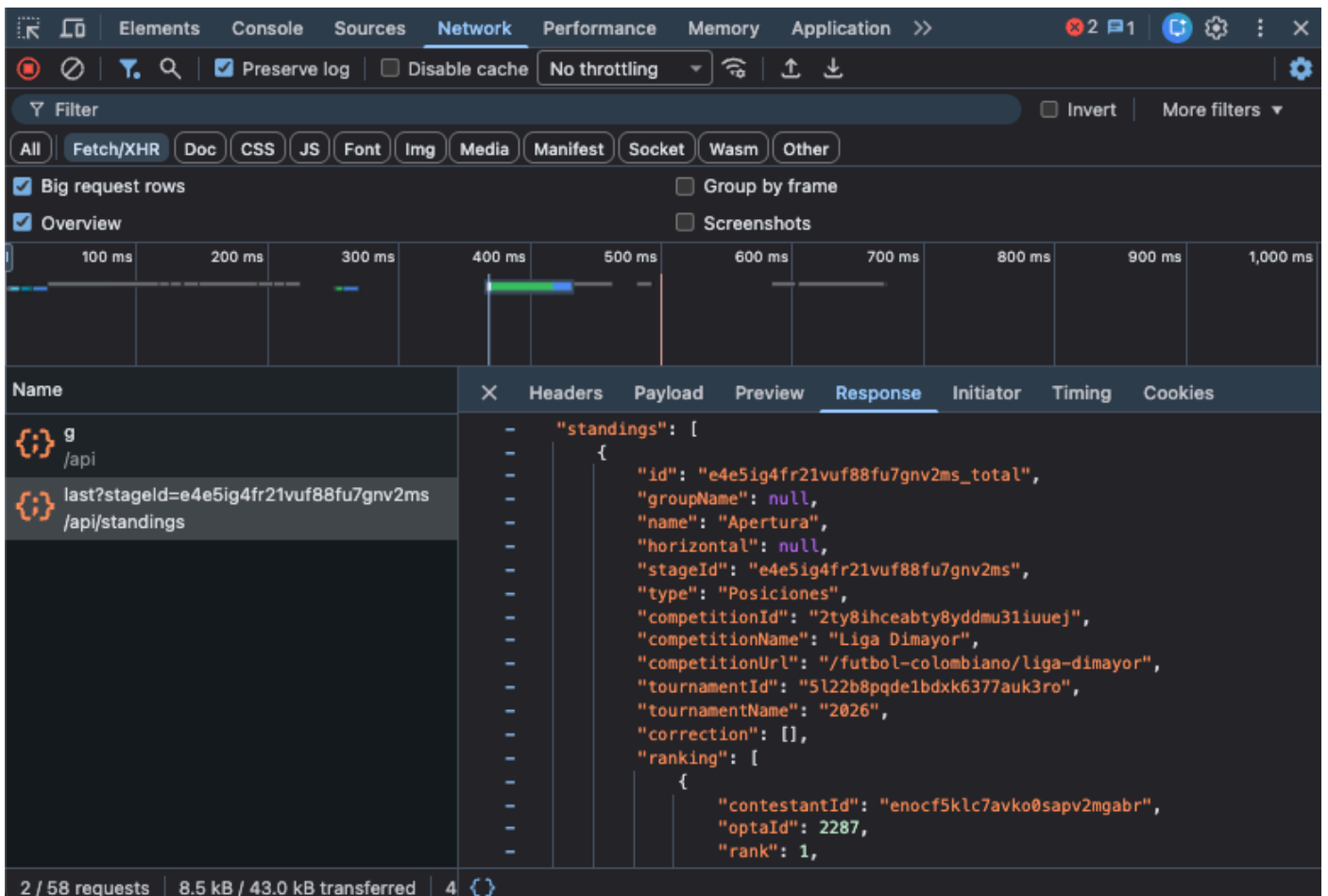
Ausencia de la tabla en el HTML inicial de Win Sports.

The screenshot shows a web browser displaying a sports website. The page has a navigation bar with links to various teams: JUNIOR, LIGA BETPLAY, ATLÉTICO NACIONAL, MILLONARIOS, SANTA FE, and AMÉRICA DE CALI. The main content area features a video player titled 'VIDEOS' and a table titled 'POSICIONES' (Positions) showing the league table for 'LIGA DIMAYOR'. The table lists teams, points, and other statistics. The browser's developer tools are open, showing the Network tab with a request to 'section.side-standing'.

EQUIPOS	PTS	PJ	DT
1 Pasto	9	4	
2 Bucaramanga	8	4	
3 Tolima	8	4	
4 América	7	4	
5 Inter Bogotá	7	4	
6 Atl. Nacional	6	2	

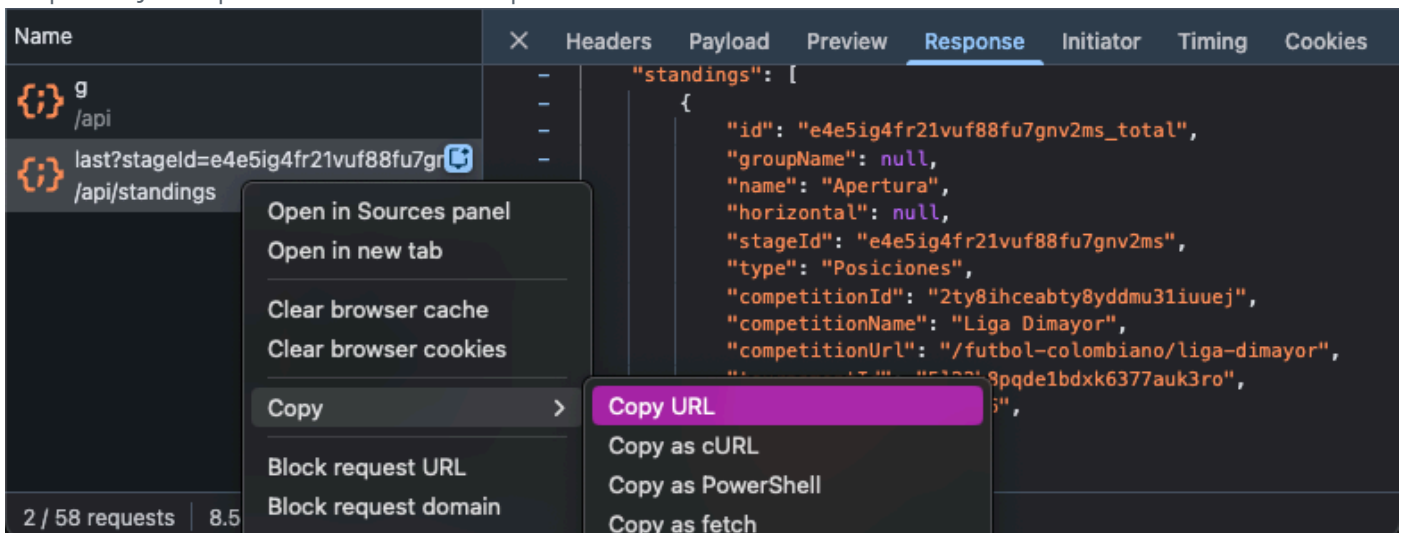
Para identificar la fuente de los datos, abrimos las herramientas de desarrollo del navegador⁵, seleccionamos la pestaña **Red** o **Network** y recargamos la página. Así podremos observar las solicitudes que realiza el navegador:

Pestaña Network de las herramientas de desarrollo.



Cada fila bajo **Name** representa una solicitud o *request*. Al aplicar el filtro **Fetch/XHR**, aparecen múltiples solicitudes. Si bien el nombre de estas no es descriptivo, el contenido de una de ellas dice *standings*. Para esta vista previa, basta con observar los valores de la pestaña **Response**.

Respuesta JSON que contiene la tabla de posiciones.



Para extraer este objeto debemos resolver dos tareas:

1. replicar con `httr` la solicitud `GET` que realiza la página, y
2. interpretar con `jsonlite` la respuesta en formato JSON (*JavaScript Object Notation*).

JSON organiza la información mediante pares de nombres y valores, listas y objetos anidados. Su estructura se parece a la de una lista nombrada en **R** o a un diccionario en Python. Este es un ejemplo sencillo:

```
{
  "people": [
    {
      "firstName": "Karl",
      "lastName": "Marx"
    },
    {
      "firstName": "Franz",
      "lastName": "Kafka"
    }
  ]
}
```

La sección **General** de la pestaña **Headers** (o encabezados) muestra el método de la solicitud (*Request Method*). En este caso, el método es **GET**.

Método GET mostrado en los encabezados de la solicitud.

httr y **httr2**

Este ejemplo conserva **httr**, que utiliza funciones como **GET()** y **content()**. El paquete **httr2** ofrece una interfaz más reciente que facilita los reintentos y el manejo de errores. Los dos paquetes permiten realizar solicitudes HTTP, pero su sintaxis no debe mezclarse dentro de una misma solicitud.

▼ Mostrar/ocultar el código

```
1 standings_url <- "https://www.winsports.co/api/standings/last?stageId=e55y43c4j8d5xjpdws5xwm"
2
3 # Reproducimos la solicitud GET observada en el navegador.
4 response <- GET(standings_url)
5
6 # Convertimos la respuesta JSON en objetos de R.
7 json_text <- content(response, as = "text", encoding = "UTF-8")
8 standings <- fromJSON(json_text)
9
```

```

10 # Extraemos el primer grupo de la tabla de posiciones.
11 standings_data <- as_tibble(
12   standings$standings$ranking[[1]]
13 )

```

Revisemos:

▼ Mostrar/ocultar el código

```
1 glimpse(standings_data)
```

Rows: 20

Columns: 19

```

$ contestantId      <chr> "8ohqbddhjvcj22wgxxvcladcx", "bzpygd5fja0tbb2x8dap...
$ optaId            <int> 2288, 2677, 2674, 7684, 2669, 7081, 2475, 2680, 22...
$ rank              <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15,...
$ rankStatus        <chr> "stt-0", "stt-0", "stt-0", "stt-0", "stt-0", "stt-...
$ contestantName     <chr> "América de Cali SA", "Deportivo Independiente Med...
$ contestantUrl      <chr> "/futbol-colombiano/equipos/america-de-cali", "/fu...
$ contestantShortName <chr> "América", "Medellín", "Tolima", "Llaneros", "Buca...
$ contestantClubName <chr> "América de Cali", "Independiente Medellín", "Depo...
$ contestantCode     <chr> "AME", "DIM", "TOL", "LLA", "BUC", "AGD", "CAL", "...
$ lastRank           <int> 1, 2, 3, 5, 4, 7, 6, 8, 9, 10, 11, 13, 12, 16, 14,...
$ points             <int> 12, 9, 9, 8, 8, 7, 6, 6, 5, 5, 4, 4, 3, 2, 1, 1, 1...
$ matchesPlayed      <int> 4, 3, 4, 4, 4, 3, 3, 3, 4, 4, 3, 4, 2, 4, 2, 3, 4,...
$ matchesWon         <int> 4, 3, 3, 2, 2, 2, 2, 2, 1, 1, 1, 1, 1, 0, 0, 0, 0,...
$ matchesLost        <int> 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 2, 1, 2, 1, 2, 3,...
$ matchesDrawn       <int> 0, 0, 0, 2, 2, 1, 0, 0, 2, 2, 1, 1, 0, 2, 1, 1, 1,...
$ goalsFor           <int> 11, 7, 8, 6, 6, 5, 4, 3, 8, 4, 5, 3, 3, 3, 0, 4, 1...
$ goalsAgainst       <int> 0, 4, 6, 3, 4, 3, 1, 1, 5, 4, 4, 5, 1, 9, 1, 6, 7,...
$ goalDifference      <chr> "+11", "+3", "+2", "+3", "+2", "+2", "+3", "+2", "...
$ lastSix            <chr> "WWWW", "WWW", "WLWW", "DWDW", "WDDW", "DWW", "WLW...

```

Efectivamente, ¡contiene la información que necesitamos! Vamos a limpiar un poco la base de datos.

▼ Mostrar/ocultar el código

```

1 columns_to_remove <- c("contestantId", "optaId", "rankStatus", "contestantUrl",
2   "contestantShortName", "contestantCode", "lastSix")
3
4 standings_data <- standings_data |>
5   select(-any_of(columns_to_remove))
6
7 standings_data |>
8   head()

```

rank	contestantName	contestantClubName
<int>	<chr>	<chr>
1	América de Cali SA	América de Cali
2	Deportivo Independiente Medellín	Independiente Medellín
3	Club Deportes Tolima SA	Deportes Tolima
4	Club Llaneros SA	Llaneros
5	CA Bucaramanga	Atlético Bucaramanga
6	Águilas Doradas	Águilas Doradas

6 rows | 1-3 of 12 columns

En este caso, los datos no estaban en el HTML inicial: identificamos la solicitud que los invocaba, reproducimos esa solicitud, y transformamos la respuesta JSON a un `tibble` de `R`.

Nota. Con este ejercicio, ¿serías capaz de recuperar la tabla de posiciones de la Champions League?

Librería `polite`

Aunque no la utilizaremos en el curso, la librería `polite` promueve una forma responsable de hacer *web scraping*. Un programa que envía demasiadas solicitudes puede sobrecargar el servidor o provocar el bloqueo temporal de la dirección IP del usuario. Nosotros, en el ejemplo de *Quotes to Scrape*, añadimos un descanso de medio segundo entre revisión de página y página, pero hay distintas formas de lograr esto.

Ejemplo de cómo no hacer *web scraping*

Imagine que quiere extraer precios de acciones de una página financiera. Un programa mal diseñado podría:

1. **Recorrer miles de páginas sin pausas.** Muchos sitios establecen límites de tasa (*rate limits*), es decir, restringen cuántas solicitudes acepta el servidor durante un periodo. Superar esos límites puede causar un bloqueo.
2. **Descargar repetidamente el mismo recurso.** Esto consume recursos del usuario y del servidor sin producir información adicional.
3. **Ignorar las reglas comunicadas por el sitio.** Debemos consultar `robots.txt` antes de automatizar las solicitudes, pues contiene los términos de uso.

Las funciones `bow()` y `scrape()` de `polite` crean una sesión y consultan el archivo `robots.txt` del *host*. Puede encontrar más información en el [sitio de la librería](#).

Recursos adicionales

- Munzert, Simon et al., 2015. Automated Data Collection with R: A Practical Guide to Web Scraping and Text Mining [\[Ver aquí\]](#)
 - Chapter 2: HTML
 - Chapter 4: Path
 - Chapter 9: Scraping the Web
 - Wickham, H., Çetinkaya-Rundel, M., & Grolemund, G. (2023). *R for Data Science*.
 - Cap. 24: Web scraping
-

Notas

1. Aplican los “disclaimers” usuales. Si tienes comentarios o sugerencias, envíanos un correo. Son bienvenidos y se considerarán en tu calificación final. [↩](#)
2. Como veremos más adelante, un *parser* es un intérprete. Un HTML es, en esencia, una cadena de texto –que, posteriormente, un navegador web interpreta y presenta de forma estilizada–. El *parser* genera una estructura tipo árbol usando, como insumo, la cadena de texto del HTML. [↩](#)
3. El *web scraping* es frágil: si cambia la estructura de la página web, el código puede dejar de encontrar los elementos esperados. Por eso, antes de hacer *web scraping*, conviene verificar si la fuente ofrece descarga directa o una API. Cuando no exista alguna de esas dos alternativas, utilizamos *web scraping*. [↩](#)
4. En el caso de Wikipedia, muchas de sus páginas redireccionan a otras páginas dentro de Wikipedia. Estas páginas suelen tener una dirección relativa. Si quisieras hacer *web scraping* de las URL, tendrías que emplear `url_absolute()` para convertir las direcciones relativas en direcciones absolutas. [↩](#)
5. Para abrirlos en Chrome, puede revisar [este enlace](#) . [↩](#)